Sprint 7 — Adapter Pattern

Class 1

Learning Objectives

By the end of this class, we will be able to:

- Explain why Adapter Pattern exists.
- Recognize incompatible interfaces.
- Implement Object Adapter.
- Understand Class Adapter.
- Apply Adapter in enterprise applications.
- Distinguish Adapter from changing existing code.

Lesson 1 — Why Adapter Exists

Start with a Real-Life Example

"Can you plug a US charger directly into a Cambodian power outlet?"

No. You need this.

US Plug -> Travel Adapter -> Cambodian Socket

The adapter doesn't change:

- the charger
- the wall socket

It simply translates one interface into another.

Exactly the same idea in software.

Software Example

Suppose our system expects every payment provider to implement:

```
public interface PaymentGateway {

    PaymentResult pay(PaymentRequest request);

}
```

Our application only understands this interface.

Unfortunately...

ABA Bank gives us this SDK.

```
public class AbaPaymentSdk {

    public AbaResponse submitPayment(
            String accountNumber,
            double amount,
            String currency
    ) {

        ...

    }

}
```

Question:

Can Java compile this?

No.

-Different interface.

-Different method.

-Different parameters.

-Different return type.

The Wrong Solution

Many beginners do this.

```
@Service
public class PaymentService {

    public void pay(...) {

        abaSdk.submitPayment(...);

    }

}
```

Tomorrow Business says: Support ACLEDA => Modify PaymentService.

Next week Support Wing => Modify PaymentService.

Next month Support TrueMoney => Modify PaymentService again.

Eventually PaymentService:

if ABA

if ACLEDA

if Truemoney

if Wing

if PayPal

...

Very ugly.

Which SOLID Principle Is Broken?

Open/Closed Principle

Every new payment provider => Modify PaymentService.

Lesson 2 — Adapter Pattern

Instead

Payment Service -> PaymentGateway -> Adapter -> External SDK

PaymentService never knows ABA , ACLEDA, TrueMoney, Wing… Only PaymentGateway

Adapter Structure

Client -> Target Interface -> Adapter -> Adaptee

Client

Our business logic. Example PaymentService

Target

Interface expected by client.

```
public interface PaymentGateway {

    PaymentResult pay(PaymentRequest request);

}
```

Adaptee

Existing class. Cannot modify.

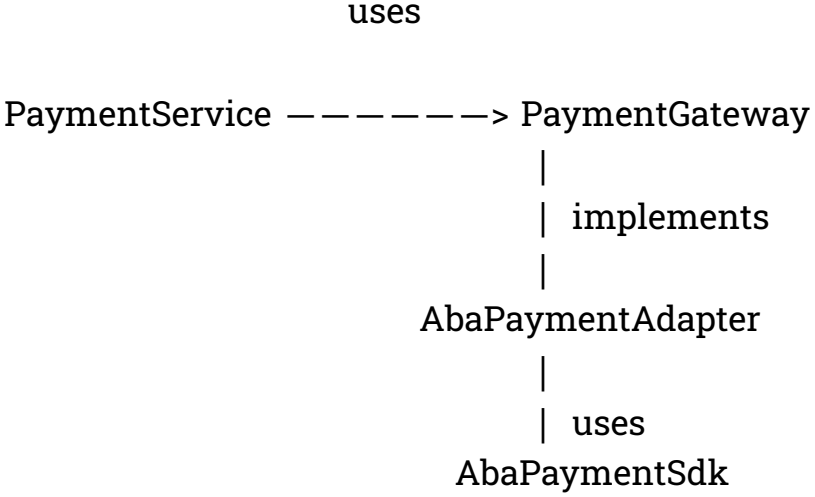
```
public class AbaPaymentSdk {

}
```

Adapter

Converts PaymentGateway -> ABA SDK

UML



Lesson 3 — Object Adapter

This is the version we should use in real projects.

Step 1

Target Interface

```
public interface PaymentGateway {

    PaymentResult pay(
            PaymentRequest request);

}
```

Step 2

Existing SDK

```
public class AbaPaymentSdk {

    public AbaResponse submitPayment(
            String account,
            double amount,
            String currency
    ) {

        ...

    }

}
```

Step 3

Adapter

```
public class AbaPaymentAdapter
        implements PaymentGateway {

    private final AbaPaymentSdk sdk;
```

```
    public AbaPaymentAdapter(
            AbaPaymentSdk sdk) {

        this.sdk = sdk;

    }

    @Override
    public PaymentResult pay(
            PaymentRequest request) {

        AbaResponse response =
                sdk.submitPayment(

                        request.account(),

                        request.amount(),

                        request.currency());

        return new PaymentResult(
                response.success());

    }

}
```

Getting point:

The adapter performs:

- Request conversion
- Method conversion
- Response conversion

Lesson 4 — Client

```
public class PaymentService {

    private final PaymentGateway gateway;

    public PaymentService(
            PaymentGateway gateway) {

        this.gateway = gateway;

    }

    public void checkout(
            PaymentRequest request) {

        gateway.pay(request);

    }

}
```

Notice: No ABA code.

Beautiful.

Demo

```
PaymentGateway gateway =
        new AbaPaymentAdapter(
                new AbaPaymentSdk());

PaymentService service =
        new PaymentService(gateway);

service.checkout(request);
```

Output

Checkout -> Adapter -> ABA SDK -> Success

Lesson 5 — Object Adapter vs Class Adapter

Most books teach both.

Most companies use only one.

Object Adapter

Adapter HAS-A SDK

Composition.

Preferred.

Class Adapter

Adapter IS-A SDK

Inheritance.

Rare in Java because Java supports only single inheritance.

Lesson 6 — SOLID Connection

OCP

Need TrueMoney?

Create TrueMoneyAdapter => Done.

PaymentService unchanged.

DIP

PaymentService depends on PaymentGateway Not ABA SDK

SRP

PaymentService Processes payment.

Adapter Converts interfaces.

SDK Communicates with bank.

Everyone has one responsibility.

Enterprise Examples

Now show students examples they already know.

Example 1

MinIO

Media Service -> ObjectStorage -> MinioStorageAdapter

Later S3StorageAdapter

Business code unchanged.

Example 2

Payment

Payment Service -> PaymentGateway ->
-ABA Adapter
-ACLEDA Adapter
-Wing Adapter

Example 3

SMS

SmsSender
-Twilio Adapter
-AWS SNS Adapter
-Local SMS Adapter

Common Mistakes

Mistake 1

Changing the SDK.

Never modify third-party libraries.

Mistake 2

Business logic inside the adapter.

Adapter should only translate.

Mistake 3

One adapter for every service.

Keep adapters focused on one external interface.

Mistake 4

Client depending directly on SDK.

Wrong: PaymentService -> ABA SDK

Correct: PaymentService -> PaymentGateway -> Adapter -> SDK

Homework

Build a Notification System.

Your application expects:

NotificationSender

Implement adapters for:

- Email SDK
- Telegram SDK
- Slack SDK
- SMS SDK

Without changing

NotificationService

Interview Questions

1. What problem does Adapter Pattern solve?
2. What is the difference between Target and Adaptee?
3. Why is Object Adapter preferred over Class Adapter in Java?
4. Which SOLID principles are supported by Adapter?
5. Can Adapter change data formats?
6. Give three enterprise examples of Adapter Pattern.
7. Can Adapter wrap a REST API or SOAP API?
8. What happens if you replace ABA with TrueMoney?

End of Class 1